



UNIVERSIDAD POLITÉCNICA DE PUEBLA
Ingeniería en Tecnologías de la Información
GESTIÓN DE DESARROLLO DE SOFTWARE

Evidencia 7

“Reporte de Pruebas”

Mariana Coatl Gutierrez

PRUEBA UNITARIA

Para esta prueba se utilizó la función de calcular el área de un hexágono regular sin usar el apotema la cual es la siguiente:

$$A = \frac{3\sqrt{3}}{2}l^2$$

Fig 1. Fórmula hexágono regular sin apotema.

Aplicación de la prueba

```
def calcular_area_hexagono(longitud_lado):  
    area = (3 * math.sqrt(3) * (longitud_lado**2)) / 2  
    return area #+ area
```

Fig 2. Función del área.

Resultado

```
1. Calcular el cuadrado de un número  
2. Calcular el área de un hexágono  
3. Salir  
Seleccione una opción (1, 2 o 3): 2  
Ingrese la longitud del lado del hexágono: 5  
El área del hexágono con longitud de lado 5.0 es: 64.95
```

FUNCIÓN CON ERROR

```
def calcular_area_hexagono(longitud_lado):  
    area = (3 * math.sqrt(3) * (longitud_lado**2)) / 2  
    return area + area
```

Fig 3. A la función se le asignó un +.

Al realizar este cambio, el resultado aumenta en cifra y por lo tanto es incorrecto.

```
1. Calcular el cuadrado de un número
2. Calcular el área de un hexágono
3. Salir
Seleccione una opción (1, 2 o 3): 2
Ingrese la longitud del lado del hexágono: 5
El área del hexágono con longitud de lado 5.0 es: 129.90
```

Fig 3. Resultado de la función.

ERROR DE DEDO

En este caso cambiamos uno de las multiplicaciones por el símbolo de la suma.

```
def calcular_area_hexagono(longitud_lado):
    area = (3 + math.sqrt(3) * (longitud_lado**2)) / 2
    return area  #+ area
```

Fig 4. Error en la multiplicación.

Resultado

```
1. Calcular el cuadrado de un número
2. Calcular el área de un hexágono
3. Salir
Seleccione una opción (1, 2 o 3): 2
Ingrese la longitud del lado del hexágono: 5
El área del hexágono con longitud de lado 5.0 es: 23.15
```

Fig 5. Cambio de resultado.

Aun utilizando el mismo número, se comprueba que el error al propósito es calculado.

ASSERTION ERROR

Para este error hicimos un assert con un valor esperado.

```
calcular_area_hexagono()  
def calcular_area_hexagono():  
    assert calcular_area_hexagono(2) == 10.39  
    assert calcular_area_hexagono(3) == 23.38
```

Fig 6. Marcación de error.

Resultado

Se observa que el error lo marca desde la línea 35 donde comienza el assert, que a pesar que se quite o corrija la siguiente línea también dará error.

```
File "/home/runner/tests/main.py", line 35  
    assert calcular_area_hexagono(2) == 10.39  
    ^  
IndentationError: expected an indented block after function  
definition on line 34  
  
File "/home/runner/tests/main.py", line 36  
    assert calcular_area_hexagono(3) == 23.38  
    ^  
IndentationError: expected an indented block after function  
definition on line 34
```

Fig 7. Registro del error.

USO DE PYTEST SIN ERROR

```
from main import calcular_area_hexagono  
  
(function) def test_hexagono() -> None  
def test_hexagono():  
    assert calcular_area_hexagono(2) == 10.39  
    assert calcular_area_hexagono(3) == 23.38  
    assert calcular_area_hexagono(-2) == 10.39  
    assert calcular_area_hexagono(-3) == 23.38  
    assert calcular_area_hexagono(0) == 0
```

Fig 8. Definición de resultados.

Para esto definimos la función test_hexágono para probar con distintos números.

Resultado

```
Seleccione una opción (1, 2 o 3): 2  
Ingrese la longitud del lado del hexágono: -3  
El área del hexágono con longitud de lado -3.0 es: 23.38
```

Fig 9. Muestra el mismo resultado.

CONCLUSIÓN

Para las pruebas se implementaron diferentes tipos de error los cuales, pueden ser de los más comunes dentro del uso de una operación o uso de la lógica, lo cual también se fue usando para aplicar las reglas de ISO en cuanto la calidad de un software, su uso, la capacidad del análisis y brindar la confianza al usuario que las herramientas son útiles o provechosas para el espacio que se necesiten utilizar. También ayuda a identificar y comprender lo que se necesita que haga el programa dentro de las advertencias de un error.